LOCKSTEP LANGUAGE

Final Scrum Project Report

Jey Lofton

Software Development Methodologies Fundamentals
Final Report

Repository: github.com/jeylofton/Lockstep-Language

1. Executive Summary

Lockstep Language is a responsive, front-end Spanish-learning website built around a structured "Tell → Show → Do" instructional method. It introduces the method, walks a visitor through how it works, previews the four language competencies it targets (Reading, Listening, Speaking, Writing), and offers a visitor-facing registration flow that simulates beginning a learning journey.

This report documents the Scrum process used to deliver the project's first sprint: the product backlog, the sprint backlog and goal, the working product increment, a sprint review demonstration plan, and a sprint retrospective, followed by lessons learned and next steps.

Scope of the completed website: a static, dependency-free HTML/CSS/JavaScript build covering six views — Home, How It Works, Learning Method Details, About Us, Register, and Confirmation — served from a single index.html with client-side routing, an interactive pronunciation/syllable demo, and front-end-only registration-form validation. There is no backend; this is explicitly a front-end prototype, documented as such on the Register page and in the project README.

2. Product Vision

Target Users

Adult and young-adult learners who want practical, everyday conversational Spanish rather than rote vocabulary memorization — learners who want to understand why a word is used, not just what it means, and apply it in real situations such as ordering food, asking for help, or making small talk.

Problem Being Solved

Many language-learning tools emphasize memorization over application. Lockstep Language addresses this by pairing every piece of vocabulary with an explanation (Tell), a demonstration of pronunciation and usage (Show), and guided practice across all four competencies (Do), so learners retain and can actually use what they learn.

Main Features

- Single-page client-side navigation across Home, How It Works, Learning Method Details, About Us, Register, and Confirmation, with the last-viewed page persisted via localStorage.
- Tell → Show → Do explainer content and the four competencies (Reading, Listening, Speaking, Writing) described across multiple pages.
- An interactive pronunciation/syllable-highlight demo with audio playback and a timed fallback if audio fails to load.
- A sample cumulative-vocabulary table illustrating spaced repetition across lessons.
- A registration form with real-time and on-submit client-side validation, inline errors, an error summary, and a show/hide password toggle.
- A responsive layout with a hamburger navigation menu below the tablet breakpoint.

Business Value

The completed prototype validates the product's core instructional concept and user flow before any backend investment is made. It gives prospective users, instructors, or stakeholders a concrete, navigable experience of the Tell → Show → Do method and the registration journey, and it establishes the front-end foundation (routing, layout system, component patterns) that a future authenticated, data-backed version would build on.

3. Product Backlog

Jira access was not available in this working session, so no Jira screenshot is included here as a matter of course — fabricating one would misrepresent the evidence. The product backlog below is the exact content prepared for entry into Jira, verified against the completed website.

ID	User Story	Priority	Points	Verified?
US-01	As a visitor, I want to understand how Lockstep Language works so I can decide whether to register.	High	3	**Yes**
US-02	As a visitor, I want to complete a registration form so I can begin my learning journey.	High	5	**Yes***
US-03	As a user, I want the website to display properly on different screen sizes so I can access it using my preferred device.	High	3	**Yes**

*Total: 11 story points. *US-02 is verified as an explicitly simulated, front-end-only registration flow — see Working Product Increment.*

US-01 — Learning Method

Description: As a visitor, I want to understand how Lockstep Language works so I can decide whether to register.

Acceptance Criteria:

- The Home page introduces the Tell → Show → Do method and the four competencies.
- A dedicated “How It Works” page walks through the three instructional stages with a worked example.
- A “Learning Method Details” page covers the full sequence, a live pronunciation/syllable demo, a cumulative-vocabulary example, and example lesson-length options.
- All of this is reachable without registering.

Related tasks: build Home, How It Works, and Learning Method Details pages; implement the pronunciation/syllable demo; wire client-side navigation.

Definition of Done: met. Verified directly in index.html (data-page="home", "how", "method") and exercised in the browser.

US-02 — Registration

Description: As a visitor, I want to complete a registration form so I can begin my learning journey.

Acceptance Criteria:

- A registration form collects first name, last name, email, password, confirm password, and Terms agreement.
- Invalid input (missing fields, bad email format, short password, mismatched passwords, unchecked terms) is caught and shown inline, with a summary error count after a submit attempt.
- A valid submission advances to a confirmation screen.

Related tasks: build the form and validation logic in scripts/script.js; add inline errors, error summary, and password-visibility toggle; build the confirmation page.

Definition of Done: met, with an important distinction — this is a simulated, front-end-only registration flow. validateForm() and the submit handler run entirely in the browser; there is no API call, server, or database. This is disclosed directly on the Register page ("Prototype only: validation runs in the browser. No data is sent anywhere.") and in the project README.

US-03 — Responsive Design

Description: As a user, I want the website to display properly on different screen sizes so I can access it using my preferred device.

Acceptance Criteria:

- Layout reflows cleanly at common breakpoints (tablet and mobile).
- Primary navigation becomes a hamburger menu on smaller screens and opens, closes on outside click, Escape, and resize.

Related tasks: establish the Flexbox/custom-properties layout system; add 860px and 560px breakpoints; build the hamburger menu and its close behaviors.

Definition of Done: met. Confirmed via the @media (max-width: 860px) and @media (max-width: 560px) rules in styles/style.css, and tested in-browser at desktop and mobile (375×812) viewport sizes.

4. Sprint Backlog

Sprint: Lockstep Language — Sprint 1

Sprint Goal: Deliver a functional, responsive website that introduces Lockstep Language, demonstrates the Tell → Show → Do instructional method, and provides visitors with a registration experience.

As with the product backlog, no Jira sprint-board screenshot is included because Jira access was not available in this session. The sprint board content below is the exact content prepared for the Jira board.

Status	Stories
To Do	(empty — all stories completed)
In Progress	(empty — all stories completed)
In Review	(empty — all stories completed)
Done	US-01 (3 pts), US-02 (5 pts, simulated flow), US-03 (3 pts)

Completed story points: 11 / 11 (100%)

All three sprint-backlog stories met their acceptance criteria against the live application and were moved to Done; none carried over.

5. Working Product Increment

The screenshots below were captured directly from the running application (python3 -m http.server) during this review, not mocked up, to demonstrate the increment as it actually behaves.



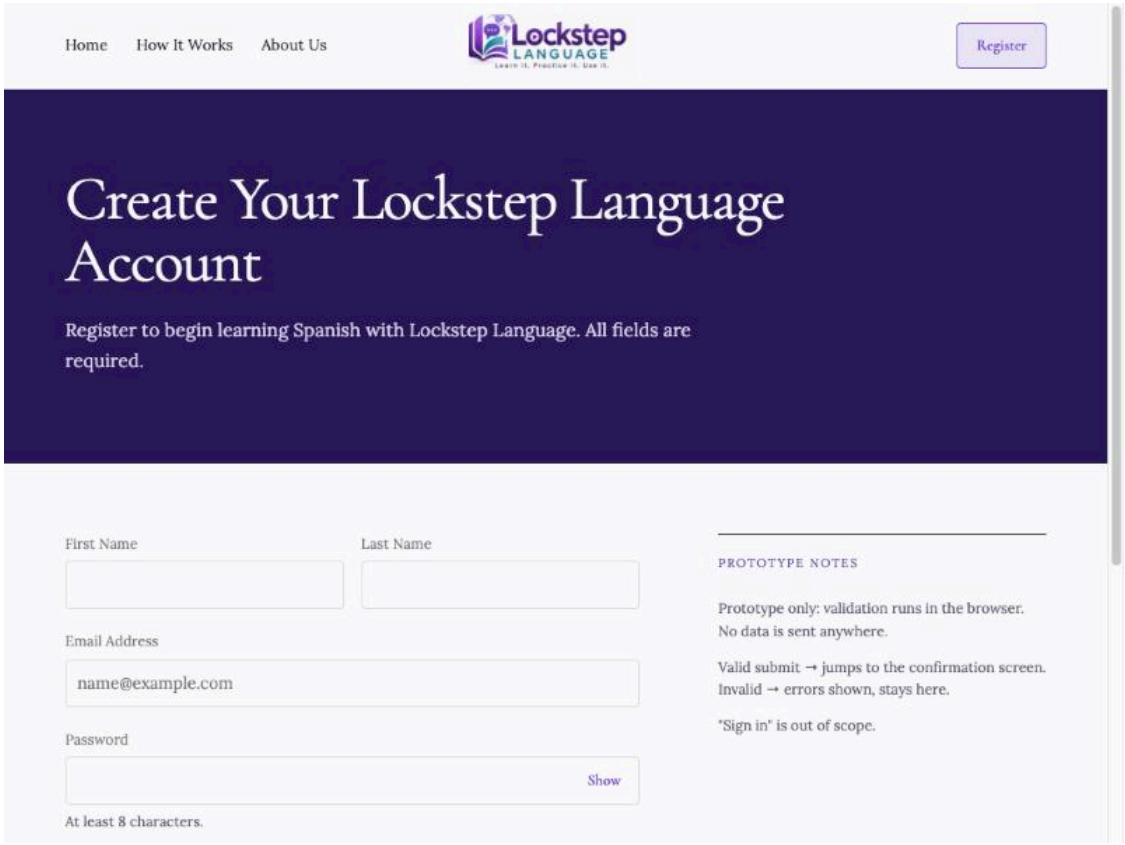
Homepage — introduces the Tell → Show → Do method.



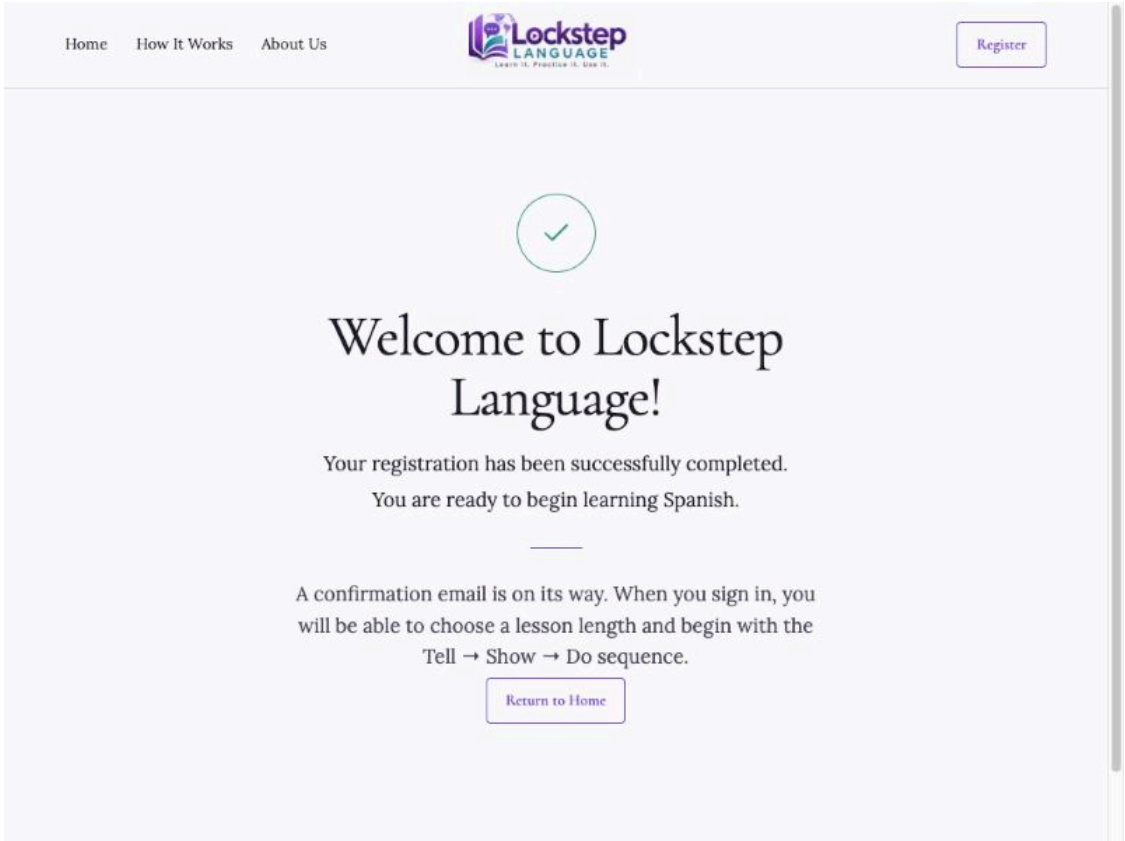
How It Works — walks through the three instructional stages.



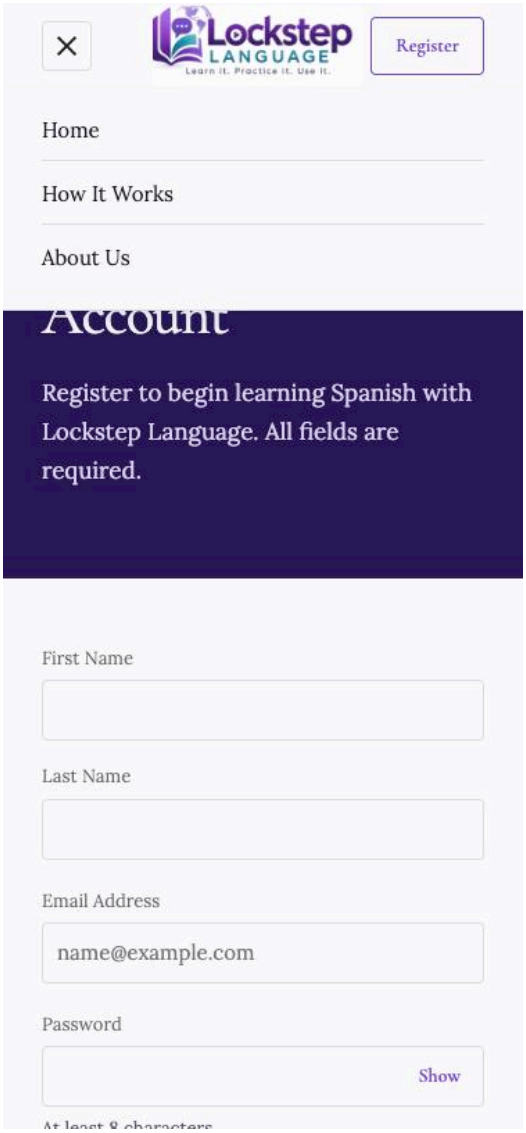
About Us — mission statement and product rationale.



Registration — client-side validated sign-up form.



Confirmation — shown after a valid registration submission.



Responsive layout — mobile hamburger navigation (375×812 viewport).

Explanation of Implemented Functionality

- Navigation: Home, How It Works, About Us, Register, and Confirmation are reachable from the header and advance via client-side routing (goToPage()) with no page reloads; the last-viewed page persists across reloads via localStorage.
- Pronunciation demo: on the Learning Method Details page, clicking a word's play button plays its MP3 and highlights each syllable in sync; if the audio fails to load, a timed highlight animation runs instead.
- Registration validation: required fields, email format, 8+ character password, matching confirm-password, and a required Terms checkbox are all enforced, with inline field errors and a form-level error-count summary.
- Confirmation flow: a valid submission transitions to the confirmation screen after a short delay, confirmed in this review using test data (see screenshot above). No account, session, or email is actually created — this is disclosed on the page itself.

- Responsive layout: the layout reflows at the 860px and 560px breakpoints in styles/style.css, and the header collapses into a hamburger menu that opens, and closes on outside click, Escape, or window resize.

Testing Evidence

This project has no automated test suite; verification is manual and functional. During this review, the following was exercised directly against the running application and confirmed working:

- Navigation across all six views, confirming content changes and active-link highlighting.
- The pronunciation/syllable demo on the Learning Method Details page.
- Registration-form validation with deliberately invalid input, confirming inline errors and the error-count banner.
- Registration-form submission with valid test data, confirming the transition to the Confirmation screen.
- Responsive behavior at a 375×812 mobile viewport, confirming the hamburger menu opens and closes correctly.

6. Sprint Review

Overview of verified completed work: all three sprint-backlog stories (US-01, US-02, US-03 — 11 of 11 story points) meet their acceptance criteria against the live application, as shown in the Working Product Increment section above. US-02 is delivered and verified specifically as a simulated, front-end-only flow.

Demonstration Plan (for the Sprint Review recording)

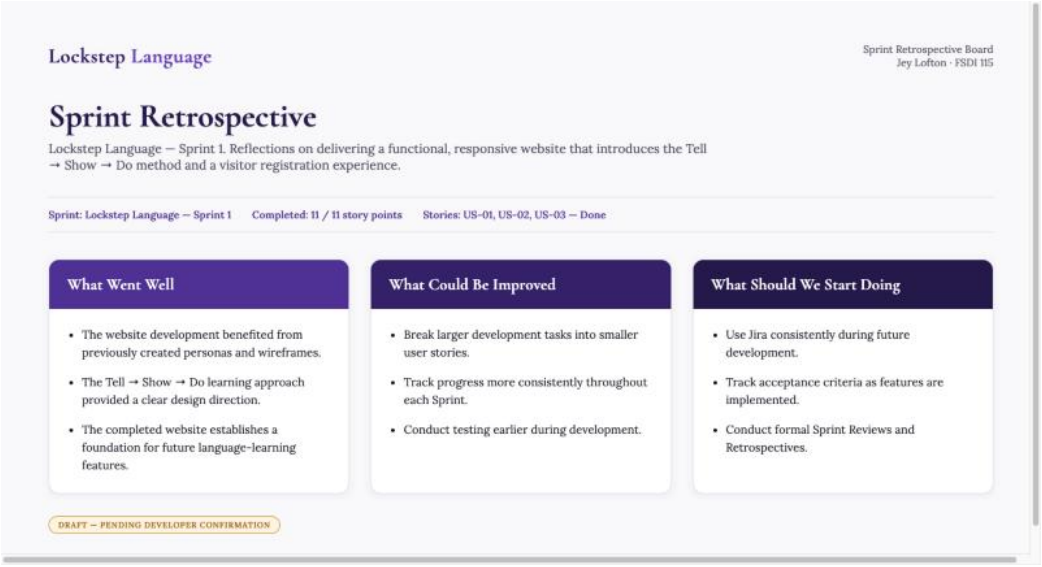
Suggested steps to record for the Sprint Review video, in order, each confirming one piece of verified functionality:

- 1. Open the site and show the Home page — narrate the Tell → Show → Do method and the four competencies.
- 2. Click to How It Works, then to Learning Method Details — play a pronunciation demo and point out the syllable highlighting.
- 3. Click to About Us — show the mission statement.
- 4. Click to Register — submit the form empty to show inline validation errors and the error-count summary.
- 5. Fill in valid values and submit — show the transition to the Confirmation screen.
- 6. Resize the browser (or use device toolbar) below 860px and 560px — show the layout reflow and the hamburger menu opening/closing.

Video Reference

No screen recording was produced during this documentation session. Recording the six steps above (roughly 2–3 minutes) is the one remaining manual step for this section — save the file as Lockstep_Language_Sprint_Review.mp4 (or similar) and reference its file location here once recorded.

7. Sprint Retrospective



Sprint Retrospective board — Lockstep Language, Sprint 1.

These points are draft reflections prepared for this report based on the completed project; they are presented here as drafted and pending Jey Lofton's confirmation, not as independently verified personal experience.

What Went Well

- The website development benefited from previously created personas and wireframes.
- The Tell → Show → Do learning approach provided a clear design direction.
- The completed website establishes a foundation for future language-learning features.

What Could Be Improved

- Break larger development tasks into smaller user stories.
- Track progress more consistently throughout each Sprint.
- Conduct testing earlier during development.

What Should We Start Doing

- Use Jira consistently during future development.
- Track acceptance criteria as features are implemented.
- Conduct formal Sprint Reviews and Retrospectives.

8. Lessons Learned

Applying Scrum as an Independent Developer

Running Scrum solo showed which parts of the framework are primarily about team communication (standups, planning meetings) and which carry their own weight even for one person — the Sprint Goal, the backlog, and the retrospective itself all did real work here by keeping scope narrow and progress visible.

Prioritizing User Stories

Writing acceptance criteria before building each page forced a clear definition of “done” for each story, which made it possible to honestly mark US-02 as Done while still being explicit that it ships as a simulated flow rather than a production registration system.

Tracking Work Using Jira

A lightweight, consistently updated board — even for a single-developer sprint — would make it easier to see what is genuinely In Progress versus Done, and would surface partially finished work (like the illustrative, non-interactive lesson-length selector) earlier rather than at review time.

Evaluating Completed Work Against Acceptance Criteria

Checking each story's acceptance criteria against the actual running application — rather than against intent — is what distinguishes “done” from “feature-complete for production.” That distinction is what allows this report to describe US-02 as verified while still being transparent about its simulated scope.

9. Conclusion

Sprint results: Sprint 1 delivered all 11 planned story points across US-01, US-02, and US-03, each verified against the live application. Nothing carried over to a future sprint.

Project outcomes: the result is a responsive, navigable front-end prototype that demonstrates the Tell → Show → Do method, an interactive pronunciation/syllable demo, and a clearly-labeled simulated registration flow — built with no framework, build step, or backend.

Future improvements, carried on the product backlog for later sprints:

- A real backend (API and database) for account creation, authentication, and sign-in.
- Real email verification in place of the simulated confirmation.
- An expanded vocabulary and audio library beyond the two current demo words.
- A functional lesson-duration selector tied to real per-user progress data.
- Automated unit and end-to-end tests.
- Consistent use of Jira for backlog and sprint tracking in future sprints.

10. GitHub Repository

Repository URL: https://github.com/jeylofton/Lockstep-Language

Source code information: the repository contains the complete static front-end build — index.html (all six page sections), scripts/script.js (routing, validation, and the audio/syllable demo), styles/style.css (layout and responsive breakpoints), plus the images/ and audio/ assets and this project's documentation under docs/. The branch is main, and at the time of this review the working tree was clean and up to date with origin/main (latest commit: “Sprint Updates”).

11. AI Disclosure

AI assistance (Claude, via Claude Code) was used in this project for documentation and preparation support: reviewing the completed codebase and cross-checking it against the sprint's acceptance criteria, capturing application screenshots from the running site, organizing the Scrum artifacts (product backlog, sprint backlog, retrospective board), and drafting this Final Report.

The Lockstep Language application itself — its design, page content, HTML/CSS/JavaScript implementation, and instructional approach — was developed by Jey Lofton. AI was not used to fabricate test results, Jira evidence, or functionality that could not be verified directly against the live site; where evidence (such as a Jira screenshot or a sprint-review recording) was not available in this session, that is disclosed explicitly in the relevant section above rather than simulated.